

QIntern 2026

Project 7 – QTagger+

Generative Augmentation and Quantum Crossover for Malware Family Classification

Report – Weeks 1 to 5

Intern: Marco Fidel Mayta Quispe

Team: Team B – Generative Augmentation & Quantum Crossover

Collaborator: Shanmukh

Project mentors: Simranjit Singh and Mohit Sajwan

Programme: QWorld QIntern 2026

Datasets: CIC-MalMem-2022 (primary); EMBER 2018 (secondary)

Period covered: Five project weeks

Final consolidation: August 2026

Declaration of Experimental Integrity

This report distinguishes between exploratory results and the results used for the final conclusions. The early notebooks are labelled as exploratory where they are discussed. The classical and fidelity results reported as final come from the Week 5 revalidation, after the protocol had been fixed. A separate quantum-classification pipeline had been run earlier and was not repeated using the locked handoff, so I report those results separately rather than combining them with the final revalidation.

There is one important limitation concerning the evaluation partition. It had already been inspected during earlier iterations of the project. For that reason, I use it here as a fixed evaluation partition for keeping the classical and augmentation experiments consistent, rather than describing it as a completely unseen external test set.

Acknowledgements

I would like to thank QWorld and the QIntern 2026 organisers for the opportunity to work on this project, as well as the project mentors for their feedback during the project. I also worked closely with Shanmukh in Team B, particularly when coordinating the augmentation work with the quantum pipeline. Discussions with members of Teams A and C were useful when defining the shared interfaces between the behavioural data, augmentation stage, and quantum classification. Some of the methodological changes described later in the report came from those discussions.

Contents

Declaration of Experimental Integrity	i
Acknowledgements	i
1 Summary	1
2 Project and Objectives	1
2.1 QTagger+ and Team B responsibility	1
2.2 Research objectives	2
2.3 Research questions	2
3 Background and Method Selection	2
3.1 Classical interpolation methods	2
3.2 Deep generative tabular modelling	2
3.3 Hybrid quantum generation	3
3.4 Evaluation dimensions	3
4 Experimental Protocol	3
4.1 Datasets	3
4.2 Locked data contract	3
4.3 Leakage controls	4
4.4 Metrics	4
5 Week 1 – Literature Review, Data Understanding, and Protocol Design	4
5.1 Activities	5

5.2	EMBER 2018 investigation	5
5.3	Week 1 deliverables	5
6	Week 2 – Classical Augmentation and Team Handoff	5
6.1	Shared artifacts and audit	5
6.2	Classical methods under the locked protocol	6
6.3	Quantum-crossover interface	6
7	Week 3 – Patch-QGAN Design and Exploratory Evaluation	6
7.1	Architecture selection	6
7.2	Minimal prototype	6
7.3	First scaled run	6
7.4	Exploratory fidelity and downstream evidence	7
8	Week 4 – QGAN Diagnosis, Stabilisation Interventions, and Controlled Evaluation	7
8.1	Diagnosis and alignment	7
8.2	Locked QGAN configuration	7
8.3	Week 4 contribution	8
9	Week 5 – Locked Revalidation and Final Consolidation	8
9.1	Full CTGAN retraining	8
9.2	Common fidelity reconciliation	8
9.3	Classical downstream benchmark	9
9.4	Quantum handoff	10
9.5	Week 5 deliverables	10
10	Integrated Discussion	11
10.1	Fidelity and utility answer different questions	11
10.2	Why generative retraining did not guarantee an advantage	11
10.3	Interpretation of the best model	11
10.4	Team-level quantum evidence	11
11	Limitations and Threats to Validity	11
11.1	Internal validity	12
11.2	Construct validity	12
11.3	External validity	12
11.4	Reproducibility limitations	12
12	Skills and Professional Development	12
13	Conclusions and Recommendations	13
A	Week-by-Week Deliverable Matrix	14
B	Code and Reproducibility	14
C	Selected Reproducibility Artifacts	14

1 Summary

The main question in this project was whether classical and quantum-assisted augmentation could help a multiclass malware-family classifier while still producing synthetic data that resembled the real observations. Most experiments used CIC-MalMem-2022, which contains benign samples together with ransomware, spyware, and Trojan samples. I also investigated EMBER 2018 as a possible secondary benchmark. In practice, its static-PE feature representation and binary task were different from the CIC-MalMem setting, and I was not able to complete the local vectorisation because of dependency incompatibilities. I therefore kept EMBER as a secondary investigation instead of forcing it into the main four-class experiment.

The work developed over five weeks. I began with the literature review and data audit, then tested classical oversampling methods, retrained CTGAN, developed and debugged a Patch-QGAN, compared synthetic fidelity, ran downstream classifiers, and prepared the quantum handoff. For the final revalidation, the source file was fixed by SHA-256. I removed 534 exact full-record duplicates, leaving 58,062 observations, and used a stratified 40,642/8,710/8,710 train/validation/evaluation split. The final pipeline retained 52 features, and preprocessing was fitted only on the real training partition.

The results did not point to one augmentation method as the best choice in every respect. SMOTE produced the closest match to the real distribution, with mean KS 0.0483, Wasserstein 0.0671, and squared RBF-MMD 0.0005. CTGAN and the QGAN were much further from the real validation data on these measures. For downstream classification, validation selected ADASYN + Random Forest, which obtained a fixed-evaluation Macro-F1 of **0.8044 ± 0.0005**. The highest score overall, however, came from Original + LightGBM, with **0.8051 ± 0.0015**. When I compared all augmentation methods against the same Random Forest baseline, ADASYN improved Macro-F1 by 0.006725, while the changes for SMOTE, CTGAN, and QGAN were all below 0.0006.

I draw three main conclusions from these results. First, synthetic fidelity and classifier performance should be evaluated separately because they did not rank the methods in the same way. Second, the generative methods did not provide a clear classical advantage under the final protocol. Third, I do not claim a quantum advantage from this work. The quantum contribution at this stage is a controlled QSVM/VQC handoff that can be evaluated in follow-up experiments.

2 Project and Objectives

2.1 QTagger+ and Team B responsibility

Project 7, QTagger+, studies classical, hybrid, and quantum approaches to malware classification. Team B, formed by Marco Fidel Mayta Quispe and Shanmukh, focused on generative augmentation and on preparing augmented tabular data for the later quantum-classification stage. My main responsibility was the QGAN work. I designed the initial version, investigated the training problems, tested stabilisation changes, and later carried out the final protocol revalidation and report consolidation.

The operational target was a four-class problem:

$$\mathcal{Y} = \{\text{Benign, Ransomware, Spyware, Trojan}\}.$$

Although CIC-MalMem-2022 is close to balanced when reduced to benign versus malicious, the four-class version is not balanced in the same way. The benign class contains roughly three times as many observations as each individual malware family, which is why augmentation remained relevant in the multiclass setting.

2.2 Research objectives

The practicum set out to:

1. review interpolation-based and deep generative augmentation for imbalanced tabular malware data;
2. build a leakage-aware train/validation/evaluation workflow for CIC-MalMem-2022;
3. compare SMOTE, Borderline-SMOTE, ADASYN, CTGAN, and a hybrid Patch-QGAN;
4. evaluate synthetic fidelity using complementary marginal and multivariate measures;
5. benchmark augmented datasets with Random Forest, XGBoost, LightGBM, and Linear SVM;
6. align reduced feature representations with QSVM/VQC resource conventions; and
7. consolidate all evidence into reproducible notebooks, manifests, figures, and handoff artifacts.

2.3 Research questions

RQ1.

Which augmentation methods most closely reproduce the real malware-family distributions?

RQ2.

Does higher synthetic fidelity translate into higher Macro-F1, MCC, or Trojan F1?

RQ3.

Can a small simulated Patch-QGAN be trained reproducibly without silently discarding failure modes?

RQ4.

Can all augmented variants be packaged under a common feature and qubit-budget convention for subsequent quantum classification?

3 Background and Method Selection

3.1 Classical interpolation methods

SMOTE creates minority-class samples by interpolating between an observation and nearby neighbours [2]. Borderline-SMOTE focuses more strongly on observations near the estimated decision boundary [3], while ADASYN allocates more synthetic samples to regions that are harder to classify [4]. These methods are inexpensive and useful as baselines. Their main weakness is that interpolation does not guarantee semantic validity: samples can be created in overlapping regions, around noisy observations, or from feature combinations that are numerically possible but not especially realistic.

3.2 Deep generative tabular modelling

I selected CTGAN as the main deep generative baseline because it was designed for tabular data with multimodal continuous variables and imbalanced discrete attributes, using mode-specific normalisation, conditional generation, and training-by-sampling [5]. TVAE and a generic cGAN were reviewed, but they were not integrated into the final experiment. During training, I used adversarial losses mainly as optimisation diagnostics. They helped identify unstable behaviour, but I did not treat a low loss by itself as evidence of good synthetic samples.

3.3 Hybrid quantum generation

The QGAN used a quantum generator together with a classical discriminator. Because the available circuit was too small to generate all retained malware features directly, I first reduced the data with PCA and reconstructed generated samples back into the original feature space afterward. This made end-to-end differentiable training possible on a simulator, but it introduced additional problems. In particular, PCA can remove information, reconstructed outputs can leave valid feature ranges, and the classical discriminator can become much stronger than the quantum generator.

3.4 Evaluation dimensions

I evaluated the methods along three separate dimensions:

- **Fidelity:** featurewise Kolmogorov–Smirnov (KS), featurewise Wasserstein distance, and squared RBF maximum mean discrepancy (MMD);
- **Downstream utility:** Accuracy, Balanced Accuracy, Macro-F1, Weighted-F1, MCC, multiclass ROC-AUC, PR-AUC, and classwise F1;
- **Quantum readiness:** reproducible reduced features, shared validation rows, matched class sizes, qubit budgets, hashes, and metadata.

PCA, UMAP, and t-SNE were used only to inspect the data visually. I did not use these plots as formal goodness-of-fit tests.

4 Experimental Protocol

4.1 Datasets

CIC-MalMem-2022, the primary dataset, was built to support memory-based detection of obfuscated malware and contains ransomware, spyware, Trojan, and benign observations [1]. EMBER, the secondary benchmark, is a large static Windows PE dataset aimed mainly at benign/malicious classification [6]. The two use different feature schemas and different task definitions, so no row-level transfer between them was attempted.

4.2 Locked data contract

Table 1: Locked CIC-MalMem-2022 data contract.

Item	Verified value
Source SHA-256	cc7a637a...4f5012a
Raw shape	58,596 rows × 58 columns
Exact full-record duplicates removed	534
Clean observations	58,062
Training / validation / fixed evaluation	40,642 / 8,710 / 8,710
Candidate numeric features / retained features	55 / 52
Split and preprocessing seed	42

I removed only exact full-record duplicates. I kept 25 files that had identical numeric vectors but came from different sources because they still represented separate observations. Stable row identifiers did not overlap between the partitions. However, 12 feature-equivalent groups, corresponding to 24 rows, were split across partitions. I report this as a limitation and recommend a group-aware sensitivity split in future work.

Table 2: Clean multiclass distribution.

Class	Count	Share
Benign	29,231	50.34%
Ransomware	9,529	16.41%
Spyware	9,815	16.90%
Trojan	9,487	16.34%
Total	58,062	100.00%

4.3 Leakage controls

The final workflow enforced the following controls:

1. split observations before imputation, scaling, dimensionality reduction, or augmentation;
2. fit constant filtering, scaling, PCA, and kernel parameters using real training data only;
3. train every augmentation method on training rows only;
4. select augmentation budgets, checkpoints, and classifier configurations with validation data;
5. keep validation and evaluation partitions fully real;
6. pair augmented and original results using the same classifier and random seed; and
7. avoid pooling numerical estimates from different splits or augmentation doses.

4.4 Metrics

For a C -class problem, Macro-F1 was computed as

$$F_{1,\text{macro}} = \frac{1}{C} \sum_{c=1}^C \frac{2 \text{Precision}_c \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}.$$

It was the primary metric because it treats every class equally, regardless of how many observations belong to it. MCC and Balanced Accuracy served as supporting metrics, and ROC-AUC and PR-AUC were computed in a one-versus-rest formulation. For Linear SVM, decision scores stood in for ranking scores; they were never described as calibrated probabilities.

On the fidelity side, lower numbers mean the distributions are closer. KS captures the largest gap between empirical marginal CDFs, Wasserstein quantifies a transport distance, and RBF-MMD measures multivariate discrepancy in a kernel-induced feature space.

5 Week 1 – Literature Review, Data Understanding, and Protocol Design

5.1 Activities

During the first week I focused on understanding the augmentation methods and setting up the experimental workflow for Team B. The literature review covered SMOTE, Borderline-SMOTE, ADASYN, cGAN, CTGAN, and TVAE. I paid particular attention to failure modes that could matter for malware data, including interpolation across class boundaries, amplification of noisy observations, mode collapse, memorisation, invalid feature combinations, and unstable adversarial training.

At the same time, I explored the primary dataset and normalised its labels to the four target classes. The early notebooks introduced the 70/15/15 stratified split, preprocessing fitted on the training set, class-distribution checks, and the initial visualisation workflow. I also decided not to judge augmentation from class counts alone. A method had to be checked for distributional fidelity, valid feature ranges, and performance on real validation or evaluation data.

5.2 EMBER 2018 investigation

I downloaded EMBER 2018 and tried to reproduce its version-2 vectorisation. The attempt stopped because of two dependency problems, the feature implementation expected LIEF 0.9.0 while the environment contained a later version, and the installed scikit-learn rejected the way a string was passed to `FeatureHasher`. There was also a task mismatch. EMBER is mainly a binary static-PE benchmark, whereas the main experiment in this project used four memory-analysis classes. For those reasons I kept EMBER as a possible secondary protocol-replication target and did not present it as a completed external validation.

5.3 Week 1 deliverables

- bilingual literature-review report and method-comparison table;
- CIC-MalMem-2022 EDA and preprocessing notebook;
- leakage-aware train/validation/evaluation protocol;
- documented EMBER environment failure and recovery options; and
- experimental plan linking fidelity, visualisation, classical classifiers, and quantum-ready representations.

6 Week 2 – Classical Augmentation and Team Handoff

6.1 Shared artifacts and audit

In Week 2, Shanmukh shared augmented datasets and notebooks for SMOTE, Borderline-SMOTE, ADASYN, CTGAN, cGAN, TVAE, together with an initial classical model comparison. These files were useful for building the first comparison workflow. Some had been generated with earlier split and balancing conventions, however, so I used them for prototyping rather than as final evidence.

The audit also revealed a provenance problem. In the early SMOTE- and CTGAN-balanced tables, real and synthetic rows were stored together without a reliable indicator of their origin. I tried to reconstruct provenance by matching rows back to the real data, but duplicate and feature-equivalent observations made that approach ambiguous. After this, the final workflow was changed to use stable row identifiers and explicit generation manifests.

6.2 Classical methods under the locked protocol

During revalidation, SMOTE, Borderline-SMOTE, and ADASYN were regenerated from locked training data only, and a fixed Random Forest evaluated on validation picked one global budget per method.

Table 3: Validation-selected classical augmentation budgets.

Method	Budget	Synthetic rows	Validation Macro-F1
SMOTE	25%	10,301	0.8071
Borderline-SMOTE	25%	10,301	0.8085
ADASYN	100%	41,202	0.8131

These budgets should be interpreted only within the selection procedure used here. Random Forest was the model used to choose them on validation data. Selecting a separate augmentation budget for every classifier would require a larger nested-validation experiment, which was outside the scope of this project.

6.3 Quantum-crossover interface

The first handoff specification included the fixed splits, retained features, target classes, synthetic-row provenance, seeds, preprocessing artifacts, and expected sample sizes. I also aligned it with Team C’s resource conventions, like PennyLane’s `default.qubit`, analytic simulation with `shots=None`, and several qubit-budget settings. This later became the basis for the canonical six-qubit, 12-feature handoff.

7 Week 3 – Patch-QGAN Design and Exploratory Evaluation

7.1 Architecture selection

I considered four QGAN designs, a single-circuit hybrid model, a Patch-QGAN, a fully quantum generator/discriminator, and a discrete-distribution qGAN. I chose the Patch-QGAN because the available circuits were too small to represent all original features directly. The first implementation therefore used two four-qubit patches that generated eight PCA components. Measurements used Pauli-Z, simulation was analytic, and training used Adam with binary cross-entropy.

7.2 Minimal prototype

I first ran a 200-row Trojan smoke test to verify that the implementation worked end to end. The generator parameters changed during training, outputs stayed finite, and checkpoints could be saved and loaded correctly. The training behaviour itself was poor. Discriminator loss fell from 0.6760 to 0.0058 while generator loss increased from 0.7751 to 5.5566. Real and synthetic samples remained visibly separated, and inverse reconstruction produced invalid negative values. At this point the implementation was functioning, but the generated data were not yet convincing.

7.3 First scaled run

I then increased the run to 2,000 Trojan rows and added several stabilisation measures, label smoothing, decaying instance noise, a weaker discriminator, two generator updates per batch, gradient clipping, early stopping, and checkpoint selection. The best checkpoint was found at

epoch 5 of a 20-epoch run. Even then, mean KS was about 0.839, Wasserstein distance was 0.459, and the synthetic-to-real diversity ratio was approximately 2.85. All raw synthetic rows violated at least one observed feature range, despite the software and gradient checks passing.

7.4 Exploratory fidelity and downstream evidence

Using the same 2,000-row Trojan reference, SMOTE showed the best early fidelity, CTGAN was intermediate, and both the raw and constrained QGAN samples were still easy to distinguish from the real data. I also ran a quick downstream benchmark using up to 30,000 real training rows. The tree-based models changed only slightly across Original, SMOTE, CTGAN, and QGAN, whereas Linear SVM was more sensitive to augmentation. I treated these results as preliminary because they came from one seed, a reduced training set, and an early QGAN configuration.

By the end of Week 3, the QGAN training pipeline was reproducible and differentiable, but the fidelity results were still poor. I therefore did not make any claim of quantum advantage from these experiments.

8 Week 4 – QGAN Diagnosis, Stabilisation Interventions, and Controlled Evaluation

8.1 Diagnosis and alignment

The training histories suggested that the main problem was discriminator dominance rather than a straightforward vanishing-gradient failure. While coordinating resource choices with Shanmukh, we also discussed related experiments in which increasing the number of qubits had reduced VQC performance. Based on these observations, I used a six-qubit analytic setup for the final revalidation, with small parameter initialisation, two data-reuploading layers, and one-dimensional ring-CNOT entanglement.

Three families of interventions were tried, across both the exploratory work and the final revalidation:

1. **small initialisation**, meant to keep the generator from starting in a displaced, high-variance state;
2. **moment regularisation**, penalising mismatches between generated and real mean and standard deviation; and
3. **training-only output-bias calibration**, shifting uncalibrated output means using statistics computed from training data.

None of these interventions solved the training problem completely. Small initialisation improved some distance metrics, but a noticeable distribution shift remained. Moment penalties reduced errors in the generated mean and standard deviation, although the full distributions still differed. Output-bias calibration corrected part of the mean shift, but it did not make the generator well calibrated on its own.

8.2 Locked QGAN configuration

The final revalidation used six qubits, eight PCA components, two re-uploading layers, analytic `default.qubit`, ring-CNOT entanglement, a maximum of 60 epochs with patience 12, generator/discriminator learning rates of 0.002/0.0005, moment penalties of 10/5, gradient clipping at 1.0, and small normal initialisation with standard deviation 0.01. A cap of 2,048 real rows per malware class kept simulator cost under control.

Training stopped at epochs 36, 35, and 33 for ransomware, spyware, and Trojan, respectively. Validation selected a 50% synthetic budget, corresponding to 20,602 rows. After generation, the domain constraints clipped about 16.68% of scaled cells. The discriminator continued to dominate for ransomware and spyware, so I refer to this model as a QGAN with stabilisation interventions rather than as a fully stabilised QGAN.

8.3 Week 4 contribution

The main outcome of Week 4 was a better controlled QGAN experiment. I added per-family training histories, moment and diversity diagnostics, constrained reconstruction, memorisation checks, checkpoint selection, and explicit stopping criteria. This made the model suitable for the final revalidation, even though its fidelity was still limited and there was no evidence of quantum advantage.

9 Week 5 – Locked Revalidation and Final Consolidation

9.1 Full CTGAN retraining

CTGAN was retrained from scratch on all 40,642 locked training rows, with independent deterministic candidates at 100, 200, and 300 epochs. Validation picked the 300-epoch checkpoint.

Table 4: CTGAN checkpoint selection on real validation data.

Epochs	Fit time (s)	Mean KS	Mean Wasserstein
100	970.7	0.3615	0.1779
200	1,778.6	0.3142	0.1491
300	2,716.6	0.3077	0.1498

The selected 300-epoch CTGAN checkpoint generated 41,202 synthetic rows at the 100% budget. Its fidelity was better than the 100-epoch candidate, but the later comparison still showed a sizeable gap relative to the real data and little downstream improvement. The longer training therefore improved the model without resolving the main limitations observed in the final evaluation.

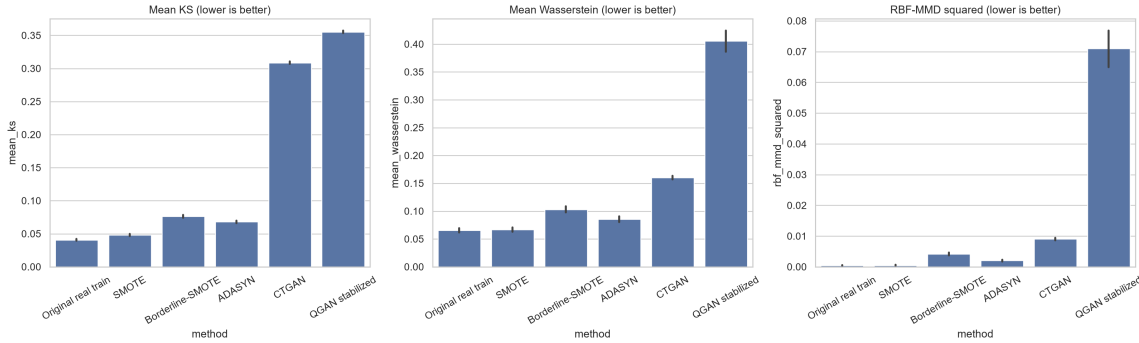
9.2 Common fidelity reconciliation

Every method was scored with one shared implementation against common real validation references, using 30 resamples of up to 512 rows per class and an RBF kernel parameter frozen from real training data.

Table 5: Common synthetic-fidelity results. Lower is better. Original is a real-real control, not an augmentation method.

Candidate	Mean KS	Wasserstein	RBF-MMD ²	Mean rank
Original real train	0.0407	0.0657	0.0004	1
SMOTE	0.0483	0.0671	0.0005	2
ADASYN	0.0680	0.0856	0.0021	3
Borderline-SMOTE	0.0765	0.1032	0.0042	4
CTGAN	0.3087	0.1601	0.0090	5
QGAN with interventions	0.3553	0.4053	0.0710	6

Among the augmentation methods, SMOTE had the best fidelity scores and QGAN had the weakest. This is consistent with the broad ordering seen in the earlier Team B exploration, but the numerical results are not directly comparable. The earlier slides used per-class median KS, whereas the final table reports global resampled means, so I keep those estimates separate.

**Figure 1:** Locked common-fidelity comparison across real and augmented candidates.

9.3 Classical downstream benchmark

Random Forest, XGBoost, LightGBM, and Linear SVM were each trained on six data variants with classifier seeds 7, 42, and 73, always against the same real validation and evaluation partitions. Validation ended up selecting ADASYN + Random Forest as the global winner.

Table 6: Validation-selected classifier per method and fixed-evaluation performance.

Method	Classifier	Macro-F1	SD	MCC	Trojan F1
Original	LightGBM	0.8051	0.0015	0.8062	0.7228
ADASYN	Random Forest	0.8044	0.0005	0.8055	0.7171
Borderline-SMOTE	Random Forest	0.8027	0.0000	0.8037	0.7178
SMOTE	Random Forest	0.7982	0.0011	0.7991	0.7129
QGAN with interventions	Random Forest	0.7981	0.0008	0.7990	0.7121
CTGAN	Random Forest	0.7981	0.0004	0.7990	0.7108

With Random Forest fixed as the paired control, the Macro-F1 changes were +0.006725 for ADASYN, +0.005056 for Borderline-SMOTE, +0.000552 for SMOTE, +0.000473 for QGAN, and

+0.000464 for CTGAN. These improvements were small. An earlier exploratory run had shown an 8.6-percentage-point gain for SMOTE, but that result used a different split and a full-balancing protocol. I therefore report it only as exploratory evidence and do not combine it with the locked estimate.

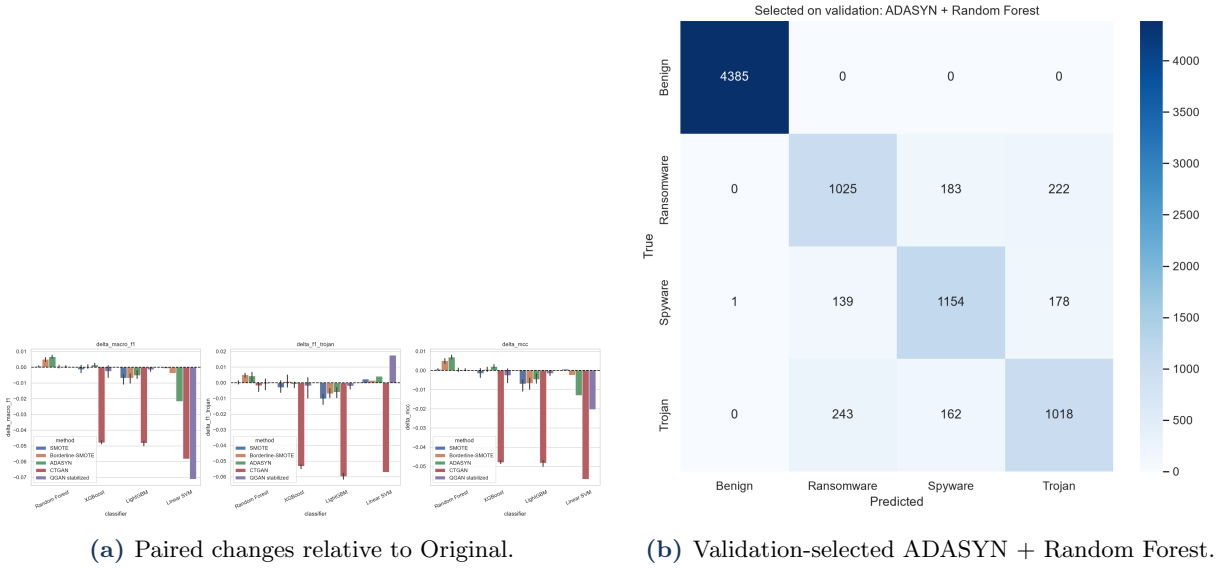


Figure 2: Downstream utility under the locked protocol.

The confusion matrix for ADASYN + Random Forest shows that benign samples were classified almost perfectly. Most remaining errors occurred between ransomware, spyware, and Trojan. Because these errors affect the malware classes unevenly, Macro-F1 and classwise F1 were more informative than accuracy alone.

9.4 Quantum handoff

The final quantum handoff contains 48 configurations, six data methods, two sample sizes (200 and 1,000), and four qubit budgets (4, 6, 8, and 12). PCA was fitted only on the original real training data, and the same validation rows are used across configurations. The fixed evaluation partition was not included in the handoff. For the canonical setting, I used six qubits and 12 features, or two features per qubit.

The handoff contains the model inputs and metadata, but it does not contain final QSVM or VQC results. Synthetic fractions differ by method because each dataset uses its own validation-selected augmentation budget. This allows the selected pipelines to be compared as complete pipelines, but it does not isolate generator choice from synthetic dose. A separate ablation with matched synthetic fractions would be needed for that comparison.

9.5 Week 5 deliverables

- locked dataset and split manifests with stable row IDs and SHA-256 checks;
- classical augmentation outputs and validation budget table;
- CTGAN candidates, checkpoints, loss histories, and selection logs;
- QGAN training histories, calibration records, constrained samples, and manifests;
- common fidelity tables, bootstrap outputs, and figures;
- full classical benchmark with paired comparisons and classwise metrics;

- 48-configuration quantum handoff; and
- final master tables, presentation slides, methodological notes, and evidence package.

10 Integrated Discussion

10.1 Fidelity and utility answer different questions

The fidelity and downstream rankings were different. SMOTE was the closest to the real data according to the fidelity metrics, while ADASYN produced the largest improvement when Random Forest was held fixed. CTGAN and QGAN were much weaker on fidelity, but their Random Forest Macro-F1 scores were still close to the baseline. In this experiment, distributional similarity was therefore not a reliable proxy for classifier improvement.

These metrics answer different questions. Fidelity measures how closely the synthetic observations match a reference distribution, while downstream evaluation measures whether the added samples help a particular classifier. The outcome can depend on the classifier, the amount of synthetic data, redundancy in the training set, and where generated samples fall relative to class boundaries. For this reason I report both types of evidence.

10.2 Why generative retraining did not guarantee an advantage

CTGAN illustrates this difference clearly. Increasing the training duration improved its validation fidelity, but the downstream classifier did not gain a meaningful advantage. The QGAN showed a similar issue from another direction, the implementation could pass gradient, checkpoint, and numerical checks while the generated distribution still differed substantially from the real one. Successful execution of a training pipeline was therefore only one part of the evaluation.

The QGAN had several additional constraints. Simulation cost limited the number of real rows used per class, PCA reduced the original feature representation, and the discriminator was consistently stronger than the generator. Output clipping was necessary to keep reconstructed values within valid ranges, but this post-processing also changed the generated distribution. These conditions should be considered when interpreting the QGAN results.

10.3 Interpretation of the best model

Original + LightGBM achieved the highest validation-selected fixed-evaluation Macro-F1, 0.8051. ADASYN + Random Forest followed closely, only 0.00065 lower. Because the experiment used one split and three classifier seeds, I do not interpret this small difference as evidence of statistical superiority. Overall, augmentation gave modest improvements in some paired comparisons, but none of the augmented pipelines clearly outperformed the strongest model trained on the original data.

10.4 Team-level quantum evidence

Other members of the team also explored QSVM and VQC models using several architectures and seeds, and some individual configurations were promising. Those experiments provide useful context for QTagger+, but they were not generated from the locked revalidation handoff used in this report. I therefore keep them separate from the controlled comparison. A follow-up experiment should run the shared handoff with multiple quantum-model seeds and paired classical baselines.

11 Limitations and Threats to Validity

11.1 Internal validity

- Twelve feature-equivalent groups cross the locked partitions, which could make evaluation slightly easier than it should be.
- The fixed evaluation partition had already been inspected in earlier iterations, so it is not a genuinely untouched external test set.
- Validation was used both to select budgets and checkpoints and, later, as the fidelity reference, a reuse that may make the fidelity estimates somewhat optimistic.
- Classifier variability rests on only three seeds and one split, so the reported standard deviations should not be read as confidence intervals.
- Neither CTGAN nor QGAN was fully repeated across several independent training seeds.

11.2 Construct validity

- KS and Wasserstein are mainly marginal measures; RBF-MMD adds multivariate sensitivity, but no single metric here proves semantic realism.
- PCA-based QGAN generation necessarily discards part of the original information.
- Clipping and rounding enforce domain validity after generation, and in doing so may shift the learned distribution.
- Synthetic fractions differ between methods because budgets were validation-selected, so architecture effects and dose effects are not fully separated.

11.3 External validity

- The main conclusions are specific to CIC-MalMem-2022 and may not carry over to behavioural traces, raw binaries, or future malware campaigns.
- EMBER 2018 uses static PE features for a different task, and its local validation was never completed.
- No physical quantum hardware was used anywhere in this work; the QGAN and the quantum handoff both rely on analytic simulation.

11.4 Reproducibility limitations

Executed notebooks are available for stages 01–07. The reporting notebook still required a final rerun after I corrected the validation-only model-selection step, and the package manifest had to be regenerated after the outputs were finalised. These issues affected the final packaging rather than the numerical experiments themselves, and they are documented here for completeness.

12 Skills and Professional Development

During the project I developed experience in several areas:

- translating a broad team plan into reproducible experimental contracts;
- diagnosing data leakage, duplicate-handling, and provenance problems;
- distinguishing optimisation diagnostics from generative fidelity;
- implementing and debugging hybrid PennyLane–PyTorch workflows;

- designing fair paired comparisons and validation-only selection;
- communicating negative and inconclusive results without overstating them;
- coordinating classical, generative, and quantum work across Teams B and C; and
- producing auditable notebooks, manifests, tables, figures, and handoff artifacts.

A major lesson from the project was that the experimental protocol sometimes had to change after weaknesses became visible. The early exploratory work exposed problems with provenance, data splits, and evaluation consistency, and the Week 5 revalidation was designed to correct those issues rather than hide them.

13 Conclusions and Recommendations

Across the five weeks, I developed and evaluated an end-to-end workflow for malware-family augmentation and quantum crossover. The work included data preparation, classical oversampling, CTGAN retraining, QGAN experiments, common fidelity evaluation, downstream classification, and preparation of a controlled quantum handoff.

The final evidence supports the following conclusions:

1. **SMOTE was the most faithful augmentation method**, though its downstream improvement under the selected 25% budget was close to neutral.
2. **ADASYN delivered the largest same-Random-Forest gain**, +0.006725 Macro-F1, while the strongest pipeline overall remained Original + LightGBM at 0.8051.
3. **Full CTGAN retraining did not fix its core weakness**, it still ranked fifth in fidelity and stayed nearly neutral with Random Forest.
4. **The QGAN is now a documented, trainable pipeline**, but discriminator dominance, clipping, and weak fidelity mean it cannot be called fully stabilised or superior to the alternatives.
5. **The locked package does not demonstrate quantum advantage**; the quantum stage is a handoff waiting on paired QSVM/VQC execution.

Several follow-up experiments would strengthen these results. I would first repeat the analysis with a group-aware sensitivity split and use separate subsets for calibration and fidelity evaluation. CTGAN and QGAN should also be trained from scratch across at least three to five seeds. The locked quantum handoff can then be evaluated with QSVM/VQC using paired classical baselines. Finally, a matched-synthetic-fraction ablation would help separate generator effects from augmentation dose, and the selected protocol should be repeated on a compatible secondary dataset with independently fitted preprocessing.

The main contribution of this project is the combination of the experimental results and the workflow used to obtain them. Synthetic fidelity, classical classifier utility, and quantum-model performance did not behave as the same quantity and should be evaluated separately. The final package provides a reproducible way to continue that comparison while keeping exploratory observations separate from the results used for the final conclusions.

A Week-by-Week Deliverable Matrix

Table 7: Summary of practicum activities and outputs.

Week	Main technical focus	Completed outputs
1	Literature, EDA, and protocol design	Method review; comparison table; CIC-MalMem audit; leakage controls; initial splits; EMBER failure diagnosis; LaTeX and Notion documentation.
2	Classical augmentation and handoff	Audited shared SMOTE/CTGAN artifacts; defined provenance-safe interface; aligned seeds, pre-processing, and quantum resource conventions.
3	Patch-QGAN implementation	Architecture review; 200-row smoke test; 2,000-row scaled run; training histories; preliminary fidelity, downstream, and ablation notebooks.
4	QGAN interventions and controlled diagnostics	Small initialisation; moment regularisation; output calibration; constrained reconstruction; validation/test diagnostics; team-alignment notes.
5	Locked revalidation and consolidation	Locked data contract; SMOTE/B-SMOTE/ADASYN generation; 300-epoch CTGAN; six-qubit QGAN; common fidelity; four-classifier benchmark; 48-case quantum handoff; master tables and presentation.

B Code and Reproducibility

The code, notebooks, experimental outputs, and reproducibility materials are available in the project repository:

https://github.com/marcofmayta/qw_q7

The repository contains the notebooks used for data auditing, augmentation, CTGAN and QGAN training, fidelity evaluation, classical benchmarking, and preparation of the quantum handoff.

C Selected Reproducibility Artifacts

Artifact	Purpose
01_data_audit_and_locked_split.ipynb	Canonical data audit, exact-duplicate removal, stable IDs, split manifest, and training-only preprocessing.
02_smote_adasyn_generation.ipynb	Generation and validation-budget selection for SMOTE, Borderline-SMOTE, and ADASYN.
03_ctgan_full_retrain.ipynb	Full-row CTGAN candidates, loss histories, checkpoint selection, and conditional sampling.

Artifact	Purpose
04_qgan_stabilized_retrain.ipynb	Per-family QGAN training, diagnostics, moment penalties, training-only calibration, and constrained output.
05_common_fidelity_evaluation.ipynb	Common resampled KS, Wasserstein, and RBF-MMD implementation.
06_classical_classifier_benchmark.ipynb	Four-classifier, three-seed benchmark with real validation and fixed evaluation.
07_quantum_handoff_builder.ipynb	Reduced feature arrays, shared validation data, qubit budgets, hashes, and handoff metadata.
08_master_tables_and_report.ipynb	Master tables, figures, written synthesis, and final package assembly.

References

- [1] Canadian Institute for Cybersecurity, “Malware Memory Analysis (CIC-MalMem-2022),” University of New Brunswick. Available: <https://www.unb.ca/cic/datasets/malmem-2022.html>.
- [2] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002. doi: <https://doi.org/10.1613/jair.953>.
- [3] H. Han, W.-Y. Wang, and B.-H. Mao, “Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning,” in *Advances in Intelligent Computing*, 2005, pp. 878–887.
- [4] H. He, Y. Bai, E. A. Garcia, and S. Li, “ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning,” in *2008 IEEE International Joint Conference on Neural Networks*, pp. 1322–1328, 2008.
- [5] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, “Modeling Tabular Data Using Conditional GAN,” in *Advances in Neural Information Processing Systems 32*, 2019. Available: <https://papers.nips.cc/paper/8953-modeling-tabular-data-using-conditional-gan>.
- [6] H. S. Anderson and P. Roth, “EMBER: An Open Dataset for Training Static PE Malware Machine Learning Models,” arXiv:1804.04637, 2018. Available: <https://arxiv.org/abs/1804.04637>.